Real-Time Video Generation With MiniMax H3

8× RTX PRO 5000 · vLLM-Omni

Shunyang Li | Public technical brief | October 2026



Agenda

01 Toward Real-Time H3 Generation
02 vLLM-Omni and ComfyUI Applications

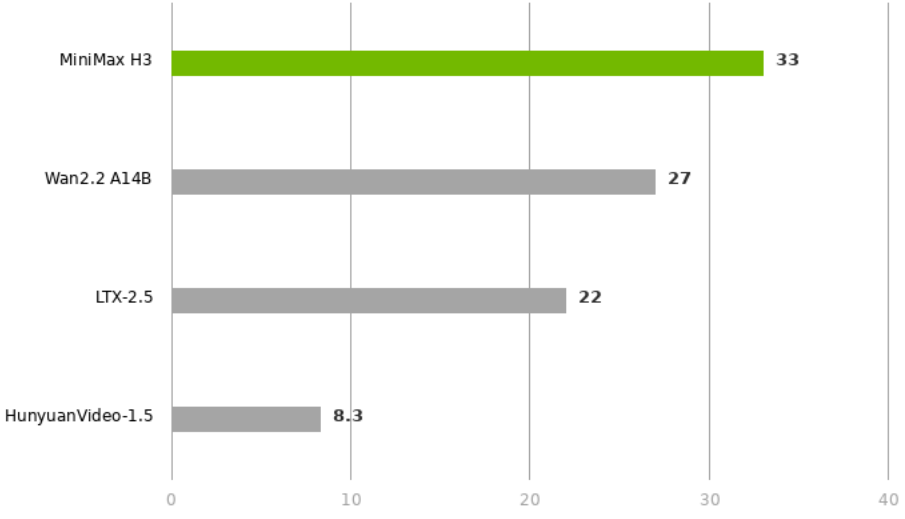


01 · Real-Time H3 Generation



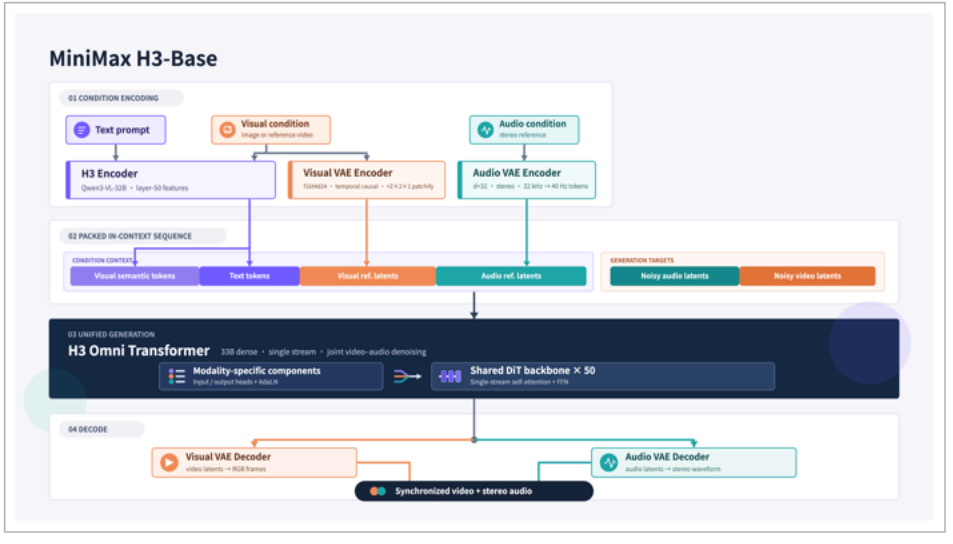
Real-Time Ref2VA Raises the Systems Bar

A 124-frame clip at 24 FPS is real time only when end-to-end latency stays below 5.167 s (RTF < 1)



Generative Backbone · Full Parameters (B)

RTF = latency ÷ media duration; the 124-frame deadline is 5.167 s. H3 has the largest dense backbone in this peer set; Wan2.2 has 27B total and 14B active parameters



Model Capability · Native 768P · 33B Dense

A 32B encoder feeds the 33B Omni Transformer and visual/audio VAEs. Fifty blocks × 50 steps dominate compute; about 13B AdaLN parameters can be cached

Sources: MiniMax H3, Wan2.2, LTX-2.5 and HunyuanVideo-1.5 official releases · author 960×544 Ref2VA workload

SM120 Brings FP4 Compute; PCIe Limits Scale

All discrete SM120 cards pair fifth-generation Tensor Cores with GDDR7 and PCIe Gen5; none provides GPU-to-GPU NVLink

SKU	Compute	Memory	GPU I/O / Fabric
RTX PRO 5000 ×8 Blackwell	SM120 · 110 SMs 5th-gen Tensor · FP4	72 GB GDDR7 ECC 48 GB variant	PCIe Gen5 ×16 Planned: RDMA via NIC*
RTX PRO 6000 WS / Server	SM120 · 188 SMs 5th-gen Tensor · FP4	96 GB GDDR7 ECC	PCIe Gen5 ×16 No GPU NVLink
GeForce RTX 5090	SM120 · 170 SMs 5th-gen Tensor · FP4	32 GB GDDR7	PCIe Gen5 ×16 No NVLink
DGX Spark · GB10	SM121 · 48 SMs 5th-gen Tensor · FP4	128 GB unified LPDDR5X	NVLink-C2C in SoC 200 Gb/s ConnectX-7

* Planned system path; GPUDirect RDMA requires a compatible NIC and PCIe topology · Sources: NVIDIA product specs, CUDA GPU table, GPUDirect RDMA guide

Latency Breakdown

NCCL occupies 49.2% of the average GPU timeline

Stage	Average GPU Time*	Share
Mean GPU timeline*	250.0 s	100.0%
NCCL communication	123.0 s	49.2%
Included: DiT collectives	116.1 s	46.4%
Included: Encoder, VAE, and other collectives	7.0 s	2.8%
Tensor compute	84.5 s	33.8%
Included: Attention	45.3 s	18.1%
Included: GEMM / MLP	39.2 s	15.7%
AdaLN weight staging + device layout	38.5 s	15.4%
Encoder, VAE, and other GPU work	4.0 s	1.6%

- Of the 38.5 s average, 36.9 s is AdaLN projection weight staging from CPU; layout is 1.5 s
- This run uses the default runtime cache, not a complete precomputed schedule sidecar

Base Ref2VA · 960×544 · 124f @ 24 FPS · 50 steps · BF16 · 8× RTX PRO 5000 · default AdaLN offload · local NCCL / RDMA off · *sum across 8 GPU timelines ÷ 8

How Denoising Works

H3 follows a learned velocity field from noise toward clean video and audio latents

State and direction

- x(t) is the current latent; the DiT predicts a data-ward velocity

Solver update

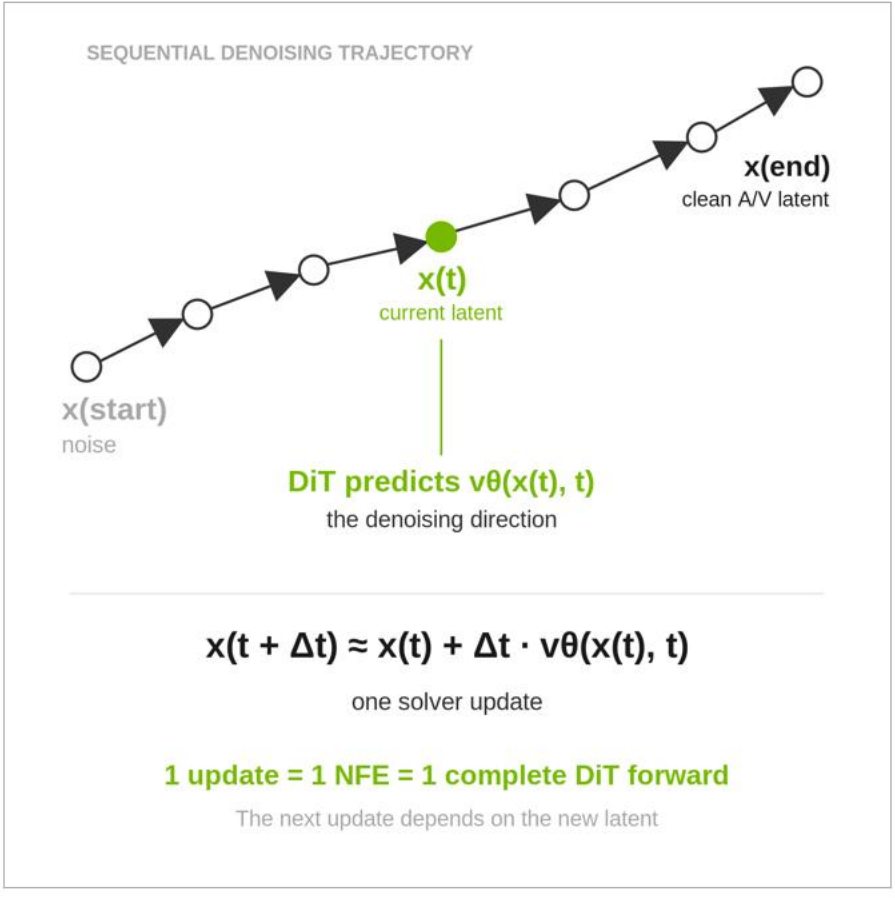
- Euler advances x(t) by a small interval along that velocity
- The next DiT call needs the new latent, so conventional steps run in sequence

Network function evaluation (NFE)

- One NFE is one complete DiT forward

Base H3

- 50 NFE × 50 main blocks = 2,500 main-block passes



Classical Flow-Matching Sampling

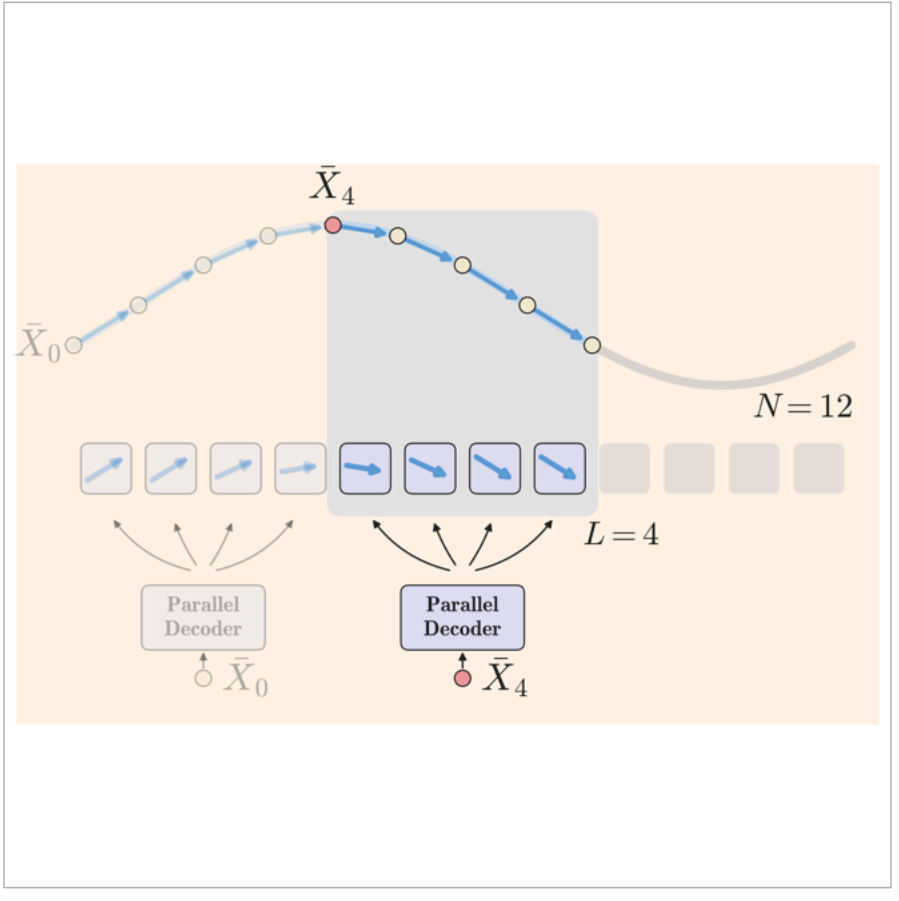
A chain of dependent solver updates turns noise into clean latents

Sources: MiniMax H3 Base configuration · NVIDIA FastGen-PDD sampling background (arXiv:2607.26004) · author abstraction

Few-Step Distillation

PDD predicts four intervals per call, reducing 50 calls to eight

- Base H3 calls the full DiT 50 times. Each call predicts one short interval, updates the latent, and provides the starting point for the next call
- PDD uses Base H3 as the teacher and trains a student on groups of four adjacent denoising intervals
- One student call predicts four consecutive directions from the same noisy latent
- The solver combines those directions, advances the latent, and invokes the student again
- The released H3 schedule covers 32 fine intervals in eight student calls
- **DiT calls fall from 50 to 8; block passes fall from 2,500 to 400 (−84%)**



N = fine intervals · L = intervals predicted in parallel

Paper example: N = 12, L = 4 · H3 PDD8: N = 32, L = 4

Sources: NVIDIA FastGen-PDD, Fig. 2 (arXiv:2607.26004) · Alibaba-PAI MiniMax-H3 Ref2VA Acc-8Step @ 335001f

Sparse Attention

Coarse attention finds relevant cubes; fine attention resolves token-level detail

Predict important regions cheaply

- Pool spatiotemporal cubes and run global attention on the shorter sequence

Select content-dependent blocks

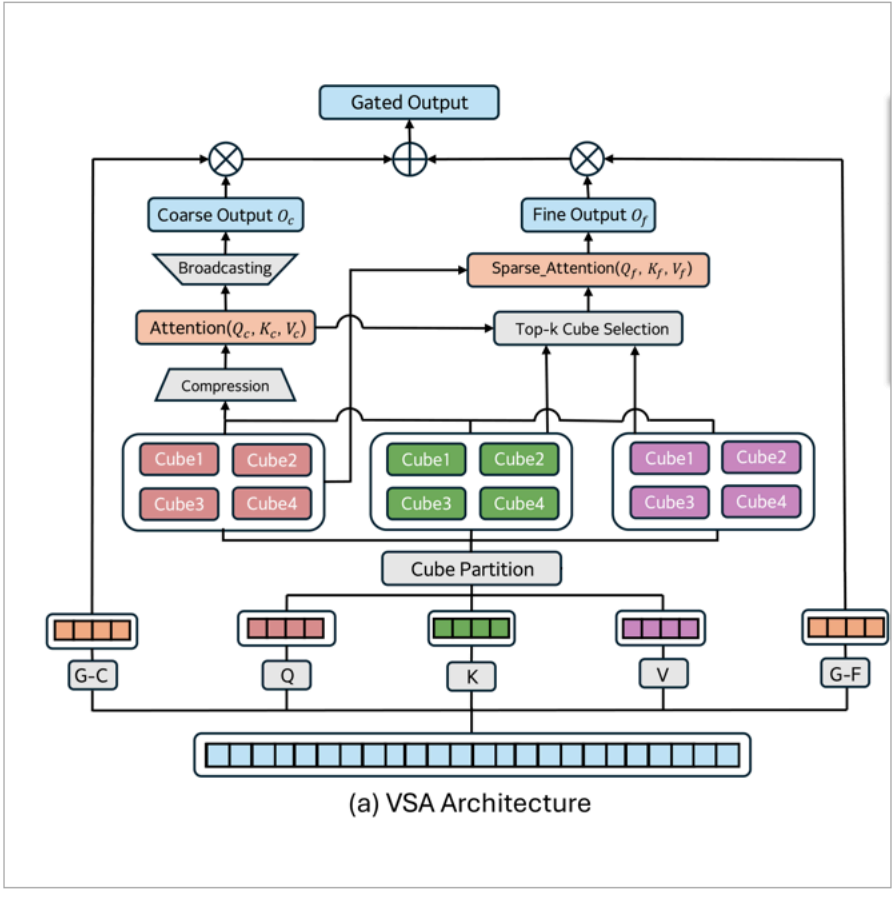
- Per-query Top-K cube scores choose which K/V regions to visit

Spend fine attention on selected detail

- Compute token attention inside the chosen blocks; skip the rest

Keep global context and local detail

- Learned gates combine coarse and fine outputs



(a) VSA Architecture

$$O = O_c \odot G_c + O_f \odot G_f$$

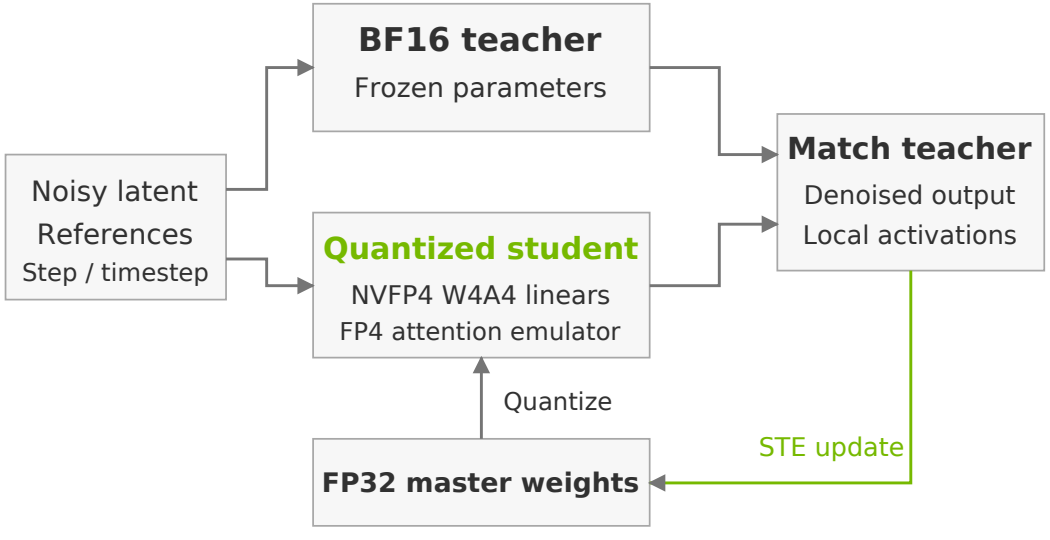
Global coarse context + selected fine detail · FastH3 adaptation: G_f = 1

Source: Video Sparse Attention, arXiv:2505.13389 · Figure 1 and Section 3 · H3 uses logical tiles of 128

FastH3 Omni: Quantization-Aware Distillation

An eight-forward student learns to compensate for quantization error before deployment

Learn with quantization in the forward pass



What runs in four bits

Module	Forward	Update
Q / K / V / O	NVFP4	Train
FFN projections	NVFP4	Train
VSA gates	NVFP4	Frozen
Norms	Original	Frozen
Boundary modules*	FP32	Frozen
Fine attention	FP4 emulator	Backprop

*Designated input / output / time modules

Execution precision ≠ trainable parameters

Few-step + four-bit inference

Eight model evaluations reduce sampling work. NVFP4 linears reduce numerical precision.

Train through quantization

FP32 master weights update through STE gradients while the forward executes NVFP4.

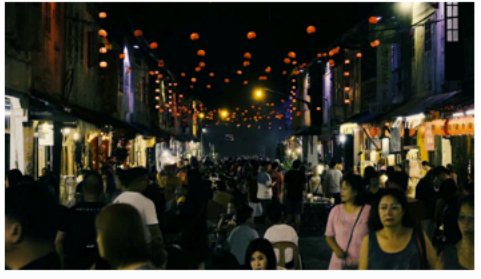
Deploy the student only

Teacher supervision and reconstruction losses recover quality during training; inference omits them.

Source: FastVideo PR #1989 · public OmniRef recipe @ ea3275d · implementation analysis, not a reproduced training result

Quantization: Crowded Night Market

Compare full quantization scopes inside vLLM-Omni with the same scene, prompt and seed

BF16 Reference	Online NVFP4	QAD NVFP4	QAD + Optimized FP4
			
BF16 weights cuDNN BF16 attention	Fresh BF16-derived NVFP4 Native FP4 sparse attention	QAD weights + native scales Native FP4 sparse attention	Same QAD encoder / decoder Optimized native FP4 attention
Request: 152.58 s*	**Request: 89.12 s***	**Request: 84.47 s***	**Request: 69.95 s***
LPIPS 0.0000 | SSIM 1.0000	LPIPS 0.2313 | SSIM 0.5488	LPIPS 0.2177 | SSIM 0.6225	LPIPS 0.2169 | SSIM 0.6222

Quantized arms include DiT, language encoder and video decoder.

1344 × 768 · 124 frames at 24 FPS · 8 steps · Seed 1234 · VSA tile128

*Single requests with CPU offload. GPUs: BF16 0; Online / QAD 3; optimized 1.

Photo: James Lo / Unsplash · one sample; no general quality claim · LPIPS lower / SSIM higher means closer to BF16

Ulysses

Each GPU swaps a token shard for seven full-sequence heads

Start with token shards

- Eight GPUs split 56,960 tokens; each rank keeps 7,120 tokens and all 56 heads

Swap the sharding axis

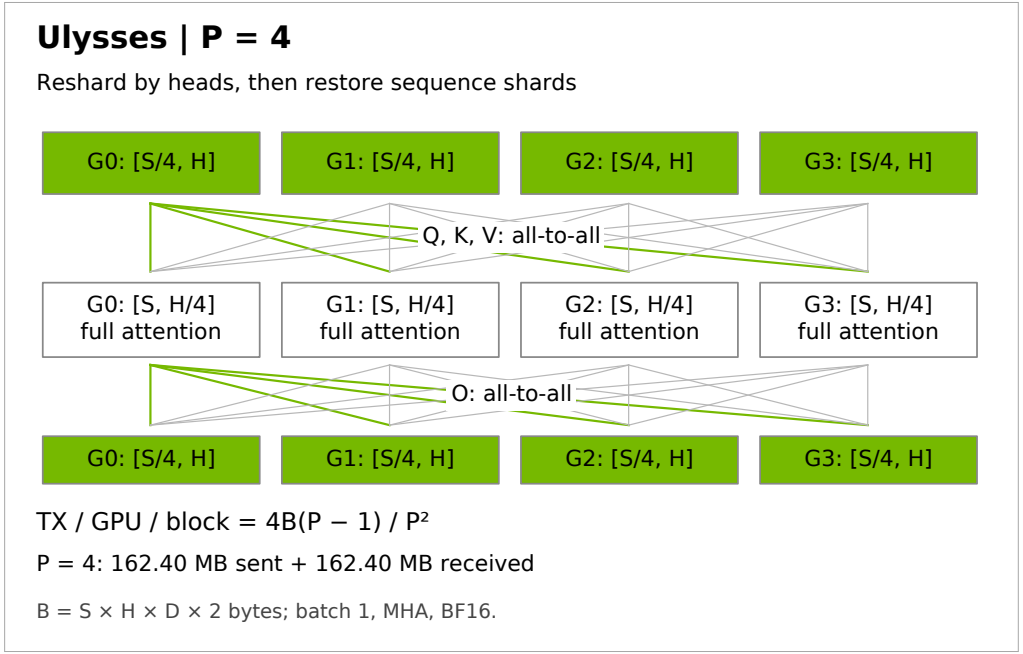
- Q/K/V/G all-to-all changes [S/8, 56] into [S, 7]

Do the expensive part locally

- Each GPU sees the full sequence for its own seven heads

Switch back for the block

- The O all-to-all restores token shards for projection, residual, and MLP



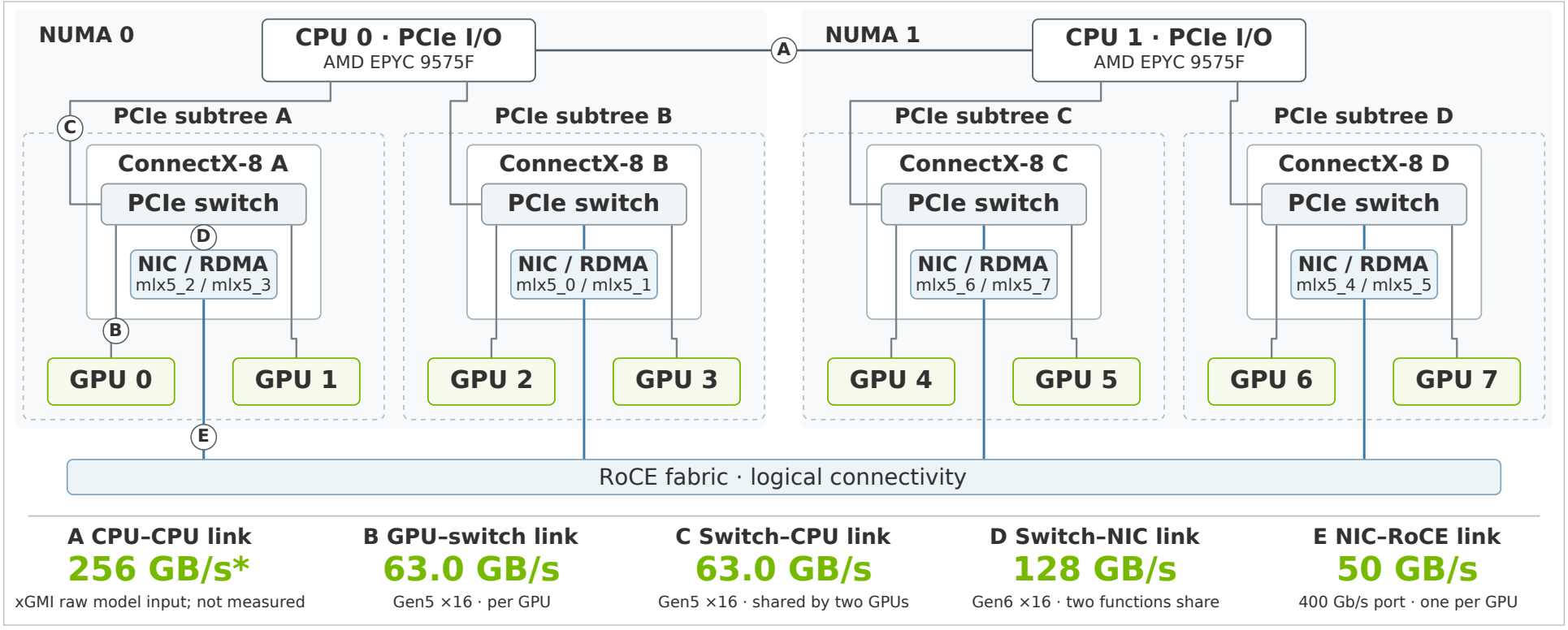
Shard by heads, then restore token shards

Original website illustration: P = 4
Measured H3 configuration at left: P = 8

Source: retained scaling website · Ulysses illustration P=4 · H3 profile at left P=8

Eight-GPU Topology

Two CPU sockets · four ConnectX-8 cards · two GPUs per PCIe subtree



One-way link capacities · network ports support simultaneous TX and RX

Sources: local PCIe / port audit · NVIDIA ConnectX-8 · AMD EPYC 9005 architecture · xGMI raw capacity assumption

Local NCCL to RDMA

Matched eight-GPU Q/K/V/O exchanges · FI RDMA is 2.33–2.47× faster across three sequence lengths

Local NCCL · shared PCIe uplink

1 P2P peer + 6 SHM peers per rank
Cross-subtree payload uses host memory
Two GPUs share one 63.0 GB/s uplink
$2 \times B \times 6/7 \le 63.0$; $B \le 36.8$ GB/s

Cross-NUMA · raw xGMI model

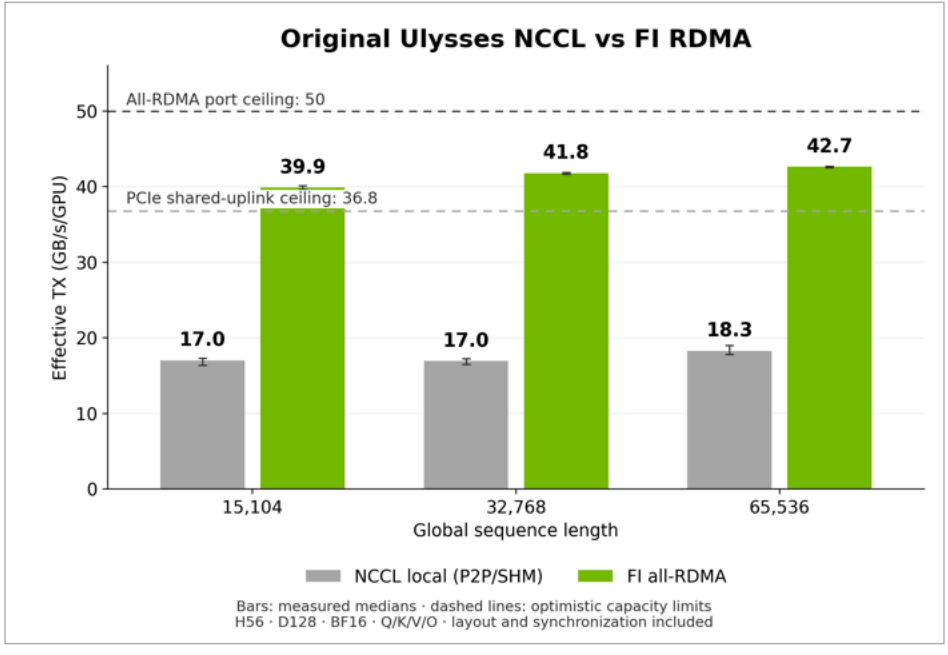
C = 256 GB/s; installed capacity is unverified
$4 \times B \times 4/7 \le 256$; $B \le 112$ GB/s

FI RDMA · nearest GPU-affine port

One nearest 400G port per GPU
Bypasses host buffers and CPU uplinks
$B \le \min(63.0, 128/2, 50) = 50$ GB/s

Shared uplink sets the tighter raw bound

Local bound: min(36.8, 112) = 36.8 GB/s
SHM, layout and scheduling add further cost
Next: 2 / 3 / 2 head bands and graph replay



Measured: communication + layout + scheduling

Separate Q/K-first candidate: 96/96 checks exact on four GPUs; full eight-GPU H3 remains pending

B: off-rank TX per GPU · C: raw one-way xGMI capacity · necessary link bounds, not measured E2E

vLLM-Omni Ulysses: NCCL Implementation

Original source: head-shard exchange before attention; inverse exchange before output projection

Q / K / V branch · comm.py

```
input_t = (
    input.reshape(bs, shard_seqlen,
                  seq_world_size, shard_hc, hs)
    .transpose(0, 2).contiguous()
)
output = torch.empty_like(input_t)
dist.all_to_all_single(
    output, input_t, group=group
)
output = output.reshape(seqlen, bs, shard_hc, hs)
output = (
    output.transpose(0, 1).contiguous()
    .reshape(bs, seqlen, shard_hc, hs)
)
```

Ulysses entry · Q / K / V

```
query = SeqAllToAll4D.apply(
    self._ulysses_pg, query,
    self._scatter_idx,
    self._gather_idx, self._use_sync
)
```

Key and value use the same call.

Ulysses exit · reverse O

```
return SeqAllToAll4D.apply(
    ctx.ulysses_pg, attn_output,
    ctx.gather_idx, ctx.scatter_idx,
    ctx.use_sync
)
```

Dense Base50: four exchanges

Q / K / V, then O. A VSA gate adds another exchange when present.

Layout work surrounds transport

Pack input, allocate output, restore layout. At B = 1, Q/K/V output can be a view.

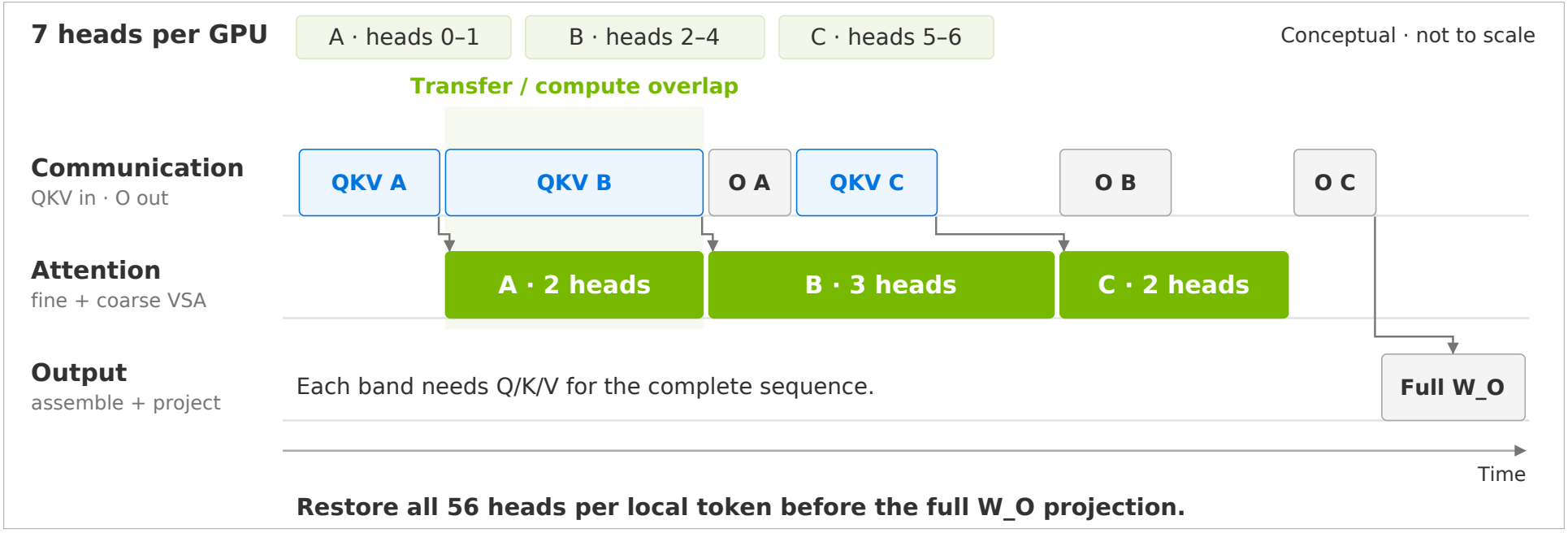
NCCL selects the physical route

P2P / SHM / NET depend on topology and runtime settings.

Source: vLLM-Omni 63487a7f22db · comm.py:44–60; ulysses.py:538–544, 721 · wrapped excerpt; guards omitted · Apache-2.0

Pipeline Ulysses by Head Bands

Use bands of 2, 3 and 2 heads per GPU to overlap communication with attention



Start earlier

Run the first two heads as soon as their full-sequence inputs arrive.

Overlap across bands

Transfer the next band while current-band attention executes.

Pipeline adds repeated submissions

Each band adds compute submissions. Graph replay becomes more valuable.

Selected schedule: 2 / 3 / 2 · dependency schematic, not a measured Nsight timeline

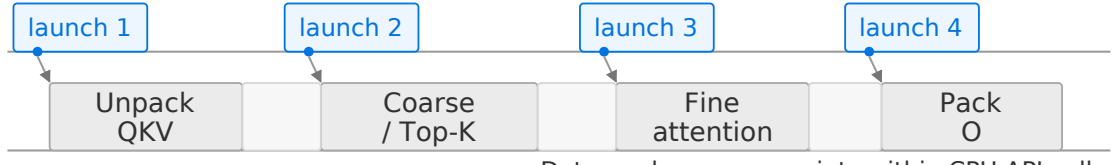
Head Bands Make CUDA Graph More Valuable

Graph replay works before banding; smaller bands increase repeated compute submission work

Same GPU kernels; fewer CPU submissions

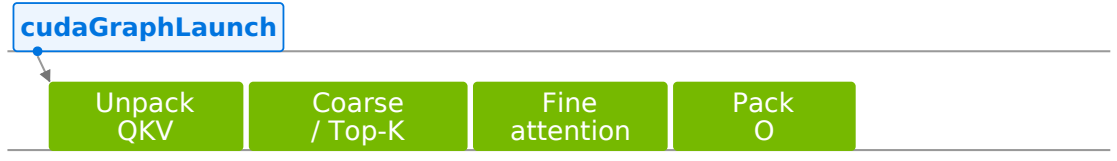
Conceptual

Eager: a launch call for each kernel



Dots mark enqueue points within CPU API calls

Graph: one replay per compute band



Replay reduces gaps when submission limits progress

GPU execution may begin before the launch API returns.

Banding strengthens the need

Each band repeats unpack, Top-K, attention and pack submissions.

A stable workload fits replay

Fixed shapes and registered buffers; tensor values may change.

400 calls amortize capture

Each band repeats over 50 blocks and eight model evaluations.

Capture compute only. RDMA, readiness events and full W_O stay outside.

Single-GPU compute experiment · historical 1 / 2 / 3 / 1 · no communication

Head layout	Eager ms	Graph ms	Reduction
All 7 heads	11.703	11.658	0.39%
Bands 1 / 2 / 3 / 1	12.323	12.040	2.30%

Banding still adds compute cost. End-to-end gains require enough communication overlap to repay it.

Retained four-arm measurements · 12 counterbalanced rounds × 20 repeats · conceptual timeline · no matched 2 / 3 / 2 result

From Baseline to Real-Time Generation

Model changes reduce work; the matched QAD systems sequence reaches RTF < 1

Base50 reference · 124 frames

247.956 s · 5.167 s of media · RTF 47.992

Model work · PDD8 + VSA + NVFP4

50 to 8 forwards; sparse attention; quantized linears

Model-stage latency deltas are unmeasured on the Base50 request; the table uses a separate 362-frame task.

Matched 15 s study · 960×544 · 362 frames / 24 FPS = 15.083 s · 8 GPUs · QAD + CAKE

Cumulative Stage	What Changes	E2E (s)	RTF
QAD + VSA + NCCL control	Eight-step NVFP4 student; original communication	17.681	1.172
RDMA + direct layout	Reduce Ulysses exchange and layout overhead	16.250	1.077
V / gate overlap	Overlap eligible work with communication	15.624	1.036
NCCL IB route enabled	Use the network route for remaining collectives	15.532	1.030
MP4 pre-encode pipeline	Overlap output encoding with generation	14.973	0.993
Production confirmation	Synchronized stage profiler disabled	14.974	0.993

Final: 14.974 s < 15.083 s of media · RTF 0.993 · 15.3% less time than the QAD NCCL control

Historical CAKE route; this is not an eight-GPU result for the optimized native FP4 variant on slide 11.
The newer 2 / 3 / 2 head pipeline and CUDA Graph results are component tests and are not included here.

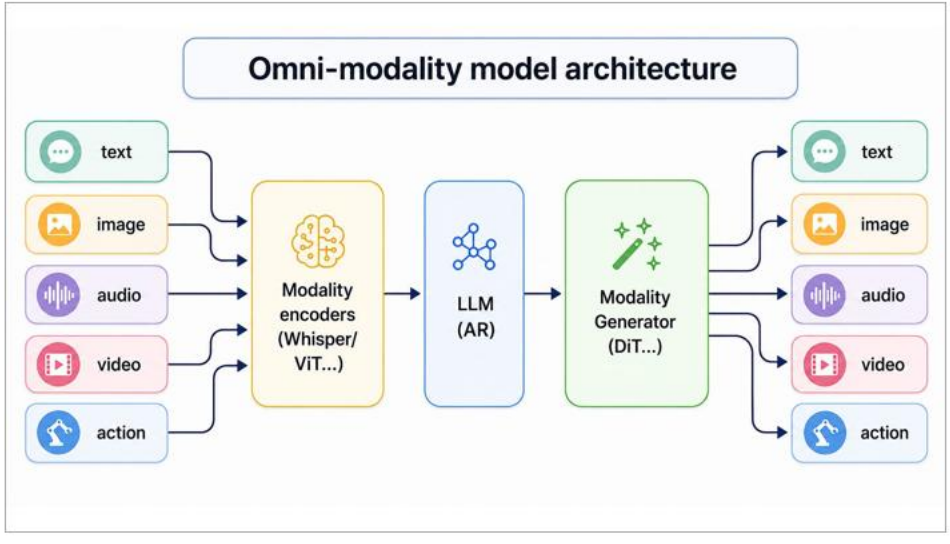
Sources: Base50 r4 · QAD 36L / CAKE study · matched warm cohorts, 3 seeds per stage · RTF = request time / media duration

02 · ComfyUI Apps With vLLM-Omni



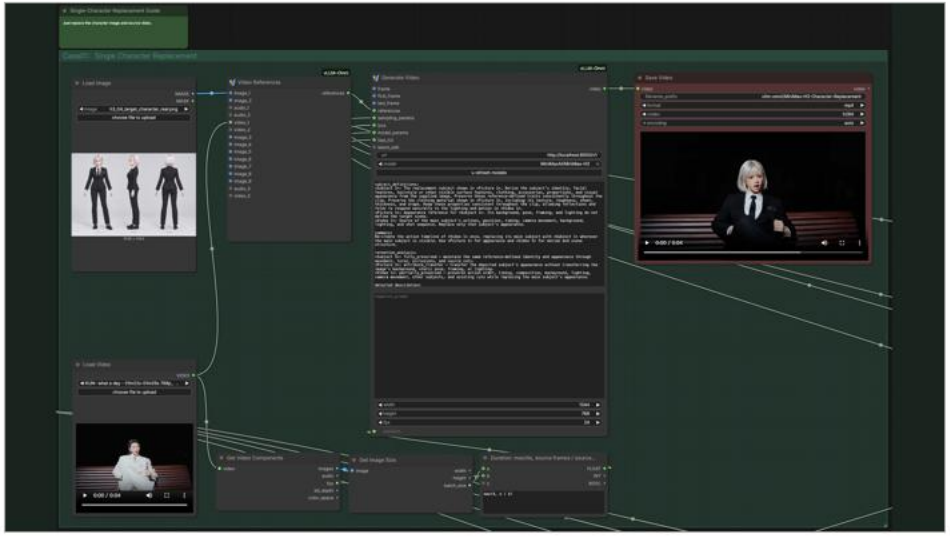
vLLM-Omni Brings Multi-GPU H3 to ComfyUI

Open PR #9194 adds character replacement, continuation-aware references, and chainable LoRA controls



Official vLLM-Omni Architecture

ComfyUI authors the workflow; vLLM-Omni runs scheduling, model execution, and multi-GPU deployment



ComfyUI Workflow · PR #9194

The workflow supports single- and multi-character replacement, source-derived duration, long-video continuation, and LoRA controls

Sources: vLLM-Omni v0.26 public deck, slide 22 · vLLM-Omni PR #9194 (open)

